

Design by Contract

Autor do Artigo: Bertrand Meyer

Autor da Resenha: Felipe Luiz Parreiras Lima

Introdução

O artigo “**Design by Contract**”, de Bertrand Meyer, apresenta uma abordagem voltada à construção de softwares mais confiáveis, corretos e robustos. O texto parte da ideia de que o desenvolvimento de software pode ser compreendido como uma sequência de decisões contratuais entre diferentes partes do sistema. Assim como em um contrato real existem obrigações e benefícios para cada envolvido, em um programa também deve haver uma definição clara sobre o que cada módulo, classe ou rotina deve receber, garantir e preservar.

O autor discute esse conceito principalmente no contexto da programação orientada a objetos e da linguagem Eiffel, que foi projetada com suporte direto a mecanismos de contrato, como pré-condições, pós-condições e invariantes. Apesar de o artigo ter uma base técnica, sua contribuição principal é metodológica: ele propõe uma forma mais disciplinada de pensar a responsabilidade entre componentes de software, evitando ambiguidades e reduzindo falhas causadas por verificações mal distribuídas ou mal documentadas.

A relevância do artigo está no fato de que muitos sistemas falham não apenas por erros de código, mas por falta de clareza sobre as responsabilidades de cada parte. Quando uma rotina é chamada, é necessário saber o que o cliente deve garantir antes da chamada e o que o fornecedor deve entregar depois. O Design by Contract surge exatamente para formalizar essa relação.

Ideia central

A ideia central do artigo é que a confiabilidade do software pode ser melhorada quando cada elemento do sistema possui um contrato explícito. Esse contrato define as condições necessárias para que uma operação seja executada corretamente e os resultados que ela deve produzir ao final.

Meyer apresenta três conceitos fundamentais: **pré-condições**, **pós-condições** e **invariantes de classe**. As pré-condições indicam o que deve ser verdadeiro antes da execução de uma rotina. Por exemplo, uma operação de remover um item de uma pilha só deve ser chamada se a pilha não estiver vazia. Já as pós-condições indicam o que deve ser verdadeiro após a execução da rotina, ou seja, aquilo que a rotina promete entregar. Os invariantes, por sua vez, representam propriedades que devem permanecer verdadeiras para os objetos de uma classe em seus estados observáveis.

O autor utiliza a metáfora do contrato para mostrar que cada lado possui obrigações e benefícios. O cliente de uma rotina tem a obrigação de respeitar suas pré-condições, mas

recebe como benefício a garantia de que, se cumprir sua parte, a rotina entregará o resultado prometido. Já a rotina tem a obrigação de cumprir sua pós-condição, mas recebe como benefício o fato de não precisar tratar casos que estão fora de seu contrato.

Um ponto importante do artigo é a crítica à chamada **programação defensiva**. Segundo Meyer, verificar repetidamente as mesmas condições em vários pontos do sistema pode aumentar a complexidade e, conseqüentemente, reduzir a confiabilidade. Para ele, não se deve atribuir a mesma responsabilidade ao cliente e ao fornecedor ao mesmo tempo. Ou uma condição pertence à pré-condição e deve ser garantida por quem chama, ou pertence à rotina e deve ser tratada por ela.

O artigo também aborda o tratamento de exceções. Meyer defende que exceções não devem ser usadas como substitutas de estruturas normais de controle, como `if` e `else`. Elas devem ser reservadas para situações realmente anormais, quando uma rotina não consegue cumprir seu contrato. Nesse caso, a rotina deve tentar restaurar um estado estável ou comunicar a falha ao chamador, sem esconder o problema.

Outro ponto relevante é a relação entre contratos, herança e ligação dinâmica. O autor afirma que, quando uma classe redefine uma rotina herdada, ela deve respeitar o contrato original. Uma subclasse pode aceitar condições mais flexíveis ou oferecer garantias mais fortes, mas não pode exigir mais do cliente nem entregar menos do que a classe original prometia. Essa visão reforça a ideia de que herança deve preservar a semântica do sistema, não apenas reutilizar código.

Aplicação em projetos reais

Em projetos reais de engenharia de software, o Design by Contract pode ser aplicado para melhorar a clareza das responsabilidades entre módulos, serviços, classes e APIs. Em sistemas corporativos, é comum que diferentes partes do sistema dependam umas das outras. Quando essas dependências não são bem especificadas, surgem erros de integração, validações duplicadas, comportamentos inesperados e dificuldade de manutenção.

Um exemplo prático ocorre em uma API de cadastro. Antes de criar um usuário, o serviço pode exigir como pré-condição que o e-mail seja válido, que o CPF esteja presente e que não exista outro cadastro ativo com os mesmos dados. Se essas condições forem responsabilidade do cliente da API, isso deve estar documentado claramente. Por outro lado, se a API promete validar tudo internamente, então essa responsabilidade passa a ser dela. O problema aparece quando as responsabilidades ficam misturadas ou implícitas.

Em sistemas com regras de negócio complexas, como plataformas acadêmicas, sistemas financeiros, sistemas de seguros ou sistemas de gestão empresarial, contratos ajudam a reduzir ambiguidades. Uma operação como “registrar nota”, “aprovar matrícula”, “confirmar pagamento” ou “gerar relatório” precisa ter condições claras de entrada e garantias claras de saída. Isso facilita a implementação, a criação de testes e a comunicação entre desenvolvedores, analistas e arquitetos.

Na arquitetura de software, a abordagem também se relaciona com boas práticas de design de componentes. Cada módulo deve expor uma interface clara e assumir responsabilidades bem delimitadas. Isso evita que uma classe ou serviço tente tratar todos os casos possíveis sem necessidade, o que normalmente gera código excessivamente complexo. Em vez disso, o sistema passa a ser organizado por acordos explícitos entre seus componentes.

Além disso, o Design by Contract pode apoiar testes automatizados. Pré-condições, pós-condições e invariantes ajudam a transformar regras de negócio em critérios verificáveis. Dessa forma, os testes deixam de validar apenas cenários pontuais e passam a verificar propriedades importantes do sistema. Isso contribui para maior segurança em refatorações, evolução de funcionalidades e manutenção de sistemas legados.

Conclusão

O artigo de Bertrand Meyer apresenta uma contribuição importante para a engenharia de software ao propor que a construção de sistemas seja baseada em contratos claros entre seus componentes. A principal mensagem do texto é que a confiabilidade não depende apenas de escrever mais verificações ou tratar todos os erros possíveis em todos os lugares, mas de definir corretamente quem é responsável por cada condição.

O Design by Contract valoriza a precisão, a simplicidade e a separação de responsabilidades. Ao utilizar pré-condições, pós-condições e invariantes, o desenvolvedor consegue documentar melhor o comportamento esperado de cada parte do sistema e reduzir ambiguidades durante a implementação. Essa abordagem também ajuda a compreender melhor temas como tratamento de exceções, herança e reutilização de código.

Como ponto crítico, o artigo tem forte ligação com a linguagem Eiffel, o que pode tornar alguns exemplos menos próximos das linguagens mais usadas atualmente. Mesmo assim, os princípios apresentados continuam atuais e podem ser aplicados em diferentes tecnologias, especialmente por meio de validações bem definidas, documentação de contratos, testes automatizados, tipagem forte e interfaces claras.

Portanto, o principal aprendizado do artigo é que sistemas robustos não surgem apenas da tentativa de prever todos os erros, mas da definição explícita das obrigações de cada parte do software. Em projetos reais, essa visão contribui para arquiteturas mais compreensíveis, manuteníveis e confiáveis, principalmente em ambientes corporativos nos quais regras de negócio e integrações precisam ser tratadas com precisão.